

A Quantitative Characterization of Semantic Conflicts in Open-Source Java Projects

Matheus Barbosa, Vinicius Resende, Ykaro dos Santos, Paulo Borba,
Rodrigo Bonifácio, Pedro Patriota, and Maria Clara Araújo
Centro de Informática, Universidade Federal de Pernambuco
Recife, Pernambuco, Brasil
{mbo2, vrb, ysa, phmb, rba2, pasp, mca}@cin.ufpe.br

Abstract—Collaborative software development relies on automated merge tools, but parallel contributions can introduce dynamic semantic conflicts that alter program behavior without triggering textual merge conflicts. Different techniques can be used to detect these conflicts, including testing, dynamic analysis, and static analysis. However, the prevalence, structural characteristics, and distribution of these semantic conflicts remain largely unexplored at scale. To address this gap, we establish a catalog of topological structures that characterize recurring structural patterns of semantic conflicts. We conduct an empirical study to quantify the occurrence of these structures in practice, using interprocedural static analysis on a large benchmark comprising 907 Java merge scenarios from open-source GitHub repositories, where the merged branches modify the same method. Our results reveal a structural dichotomy: while CF conflicts appear mostly in fully local topologies (90.4% start and end in the same file), DF and OA conflicts appear in multi-file topologies far more often, accounting for 60.3% and 46.7% of their respective instances. These findings provide actionable guidance for configuring static analysis tools, demonstrating that fast intraprocedural analysis is sufficient to detect most CF conflicts, whereas deeper interprocedural analysis might be essential to uncover DF and OA conflicts.

Index Terms—Semantic Conflict, Static Analysis, Merge Tools, Direct Flow (DF), Confluence Flow (CF), Override Assignment (OA)

I. INTRODUCTION

Collaborative software development depends on developers continuously merging their contributions into shared repositories. Modern version control systems automate most of this integration through text-based merge tools such as `diff3` [1], which detect and report textual conflicts when developers modify the same or consecutive lines of code. Despite the maturity of these textual merge algorithms, a class of subtle integration errors goes completely unnoticed: *dynamic semantic conflicts* [2], [3] occur when parallel modifications from two branches, each individually correct, produce unintended behavior in the merged program.

Previous studies have explored three static analyses that detect common patterns of interference.¹ A *Direct Flow* (DF) conflict arises when one branch writes a state element and the

other branch reads that state element.² A *Override Assignment* (OA) conflict happens when both branches write to the same state element with no intervening base write between them. A *Confluence Flow* (CF) conflict occurs when two branches independently alter separate state elements that jointly influence a common computation, thereby distorting the merged result. None of these conflicts triggers a textual merge conflict, and they easily slip past code review and can corrupt production behavior.

To detect semantic conflicts before they reach production, researchers have developed static analysis tools [4]. These tools operate on three-way merge scenarios and traverse program call graphs to track which state elements are written and read across branches. However, while these analyses are able to detect semantic conflicts in real-world projects, a fundamental gap remains: the *prevalence*, *structural characteristics*, and *topological distribution* of the conflicts detected by each analysis have not been quantitatively characterized. Without this empirical knowledge, designers of static analysis tools lack the data needed to calibrate analysis depth, balance detection recall against computational cost, and prioritize the most impactful conflict patterns.

Understanding the structural characteristics and prevalence of semantic conflicts is important for three reasons. First, recognizing the most recurrent patterns helps developers and integrators anticipate and avoid risky merge scenarios. Second, empirical knowledge of these conflicts allows researchers to direct future studies more effectively and explore novel alternatives for conflict resolution. Third, similar to how catalogues of build conflicts enable the development of program repair and awareness tools [5], a topological classification of semantic conflicts can guide the creation of automated repair techniques, advanced code review tools, and proactive warnings in modern development environments. Ultimately, this foundational knowledge is essential to optimize future static analyses.

To address this gap, we conduct a quantitative characterization of DF, CF, and OA semantic conflicts. First, we establish

¹Changes L and R from separate branches to a base program B interfere when the integrated changes do not preserve the changed behavior of L or R , or the unchanged behavior of B .

²State elements refer to program entities that hold or represent the state of a program during its execution. These include global variables, object fields, system files and streams, variables that store method return values, raised exceptions, and any other entities that maintain state information across execution contexts.

a catalog of 54 topological structures that capture recurring topologies of these conflicts based on file boundaries and call-graph shape. We then run interprocedural static analysis on the RefDataset [6], a large benchmark comprising 907 Java merge scenarios from open-source GitHub repositories, where the merged branches modify the same method. For each detected conflict, we extract the originating and terminating nodes of the conflicting execution path, abstract chained calls and calls within the same file, and match the resulting structure against our catalog. We then quantify the frequency of each structure across all detected conflicts. For example, these structures capture whether the conflicting nodes are completely contained within a single file or whether they span multiple distinct files. In doing so, they formalize the number of files involved, the relative locations of the conflicting nodes, and the interprocedural pathways through which independent developer changes ultimately collide. Since the dataset restricts to merge scenarios where the merged branches modify the same method, the conflicting changes necessarily originate in the same file; however, the analyses itself remains interprocedural and tracks conflict paths that traverse multiple methods and files.

Our results reveal a structural dichotomy in how semantic conflicts manifest: while CF conflicts concentrate in intra-file topologies (90.4% belong to structures that start and end in the same file), both DF and OA conflicts exhibit highly divergent behavior, with topologies that start in the same file but end in different files accounting for 60.3% of the 31,747 DF instances and 46.7% of the 105 OA instances. These findings provide actionable guidance for configuring static analysis tools in daily software integration pipelines.

This paper makes the following **contributions**:

- 1) A catalog of topological structures that define how DF, CF, and OA semantic conflicts manifest across file boundaries and call-graph structures.
- 2) A quantitative characterization of the prevalence and file-level distribution of 32,361 semantic conflict instances across 907 real-world merge scenarios.
- 3) Actionable guidance for configuring static analysis tools, balancing detection recall against computational cost in daily software integration pipelines.

II. SEMANTIC CONFLICTS

To detect semantic conflicts that escape textual merge tools, researchers have formalized three analyses based on how parallel modifications interact with shared program state. This section defines these three analyses, following the formalization established by Santos de Jesus et al. [4], and illustrates each with a concrete code scenario.

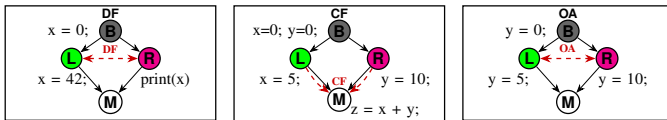


Fig. 1. Canonical DF, CF, and OA conflict patterns.

A. Direct Flow Conflicts

A DF conflict can result from an interference in which one branch writes a program state element and the other branch reads it, including flows through expressions in control structures and return statements.

Figure 1 (DF) illustrates the canonical DF pattern. In the base version, state element x holds the value 0. The Left branch assigns $x = 42$, while the Right branch introduces a `print(x)` statement. After merge, because the assignment precedes the `print` call in the merged method body, the `print` statement reads the value 42, a value that the Right-branch developer never anticipated. The merged program compiles and runs without errors, yet its behavior differs from what either developer intended.

Listing 1 presents a more realistic scenario involving a configuration class (in this and all subsequent code listings, lines modified by the Left branch are highlighted in green, and lines modified by the Right branch are highlighted in pink). Developer Left modifies a default timeout value, while Developer Right adds a new method that reads the same field to compute a retry interval. After the merge, the retry interval silently doubles because the timeout constant changed.

```
class RetryPolicy {
    int timeout = 30 60; // Left branch:
                        doubled for slow networks
    int interval(Config c) {
        return c.timeout * 2; // Right
                             branch: expects 60, gets 120
    }
}
```

Listing 1. DF conflict: Developer Left changes a timeout constant; Developer Right reads the same field to compute a retry interval.

B. Override Assignment Conflicts

An OA conflict can result from an interference in which both branches write to the same program state element, with no intervening base-version write, so the merge silently discards one of the updates.

```
class Logger {
    // Left branch: changes default level
    Level level = Level.INFO; Level.DEBUG;

    void init() {
        loadConfig();
        // Right branch: overrides level
        level = Level.WARN;
    }
}
// Merged: init() assignment silently
// overwrites the default
```

Listing 2. OA conflict: both branches independently set the logging level; the merge silently discards one assignment.

Figure 1 (OA) depicts the canonical OA pattern, where both branches write y and one write is silently lost. Listing 2 demonstrates a practical OA scenario.

C. Confluence Flow Conflicts

A CF conflict can occur when there is no direct flow between the integrated changes, but there are direct flows from the integrated changes to a common statement; this statement’s behavior might then be affected by both changes, characterizing an interference. Figure 1 (CF) illustrates the canonical CF pattern. In the base, both x and y equal 0. Developer Left sets $x = 5$; Developer Right sets $y = 10$. After the merge, because both assignments precede the expression $z = x + y$ in the control flow of the merged method, the expression evaluates to 15, a value that neither developer expected, since each assumed the other state element remained at its base value.

Listing 3 shows a practical CF scenario involving a pricing computation.

```
class Invoice {
  // Left branch: increases discount
  double discount = 0.10 0.20;;
  // Right branch: increases tax
  double tax = 0.05 0.12;;
  // Merged: finalPrice combines both
  // changes
  double finalPrice(double base) {
    return base * (1 - discount) * (1 +
      tax);
  }
}
```

Listing 3. CF conflict: Developer Left changes the discount rate; Developer Right changes the tax rate; the final price reflects both changes simultaneously.

III. EMPIRICAL STUDY SETUP

This section describes the setup of our study. We first formulate the research questions guiding our investigation. Next, we explain how we define the topologies that capture the structural patterns of semantic conflicts. We then detail the dataset and describe the study procedures, including our interprocedural static analysis approach and instrumentation. Figure 2 provides an overview of the empirical evaluation pipeline.

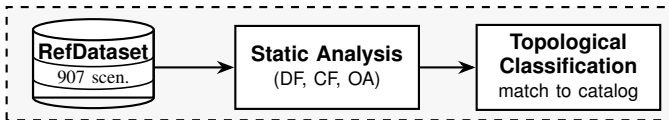


Fig. 2. Workflow of the empirical study.

A. Research Questions

We formulate two research questions to guide our empirical characterization:

RQ1 What are the possible topological patterns in which DF, CF, and OA semantic conflicts manifest, and what properties define each structure?

RQ2 How many conflicts of each topological structure occur in real-world open-source Java merge scenarios?

RQ1 presents and explains the catalog of topological structures for each conflict type, providing a visual and structural understanding of how semantic conflicts materialize across files, methods, and call graphs. RQ2 quantifies the occurrence of each structure by executing the static analysis on the RefDataset, revealing which patterns dominate in practice and which remain rare.

B. Topological Structure Definition

Before executing the large-scale experiment, we established a foundational scientific contribution of this work: the systematic definition and cataloging of topological structures for DF, CF, and OA conflicts. To derive these structures, we employed an empirical and analytical research method based on our knowledge and experience with the outputs of static analyses. First, in an exploratory phase, we observed the raw outputs of static analysis executions on a subset of merge scenarios and tested toy examples (see our online appendix [7]) to extract initial, empirically grounded conflict structures. Next, we systematically derived additional permutations by extrapolating these base cases along the dimensions of file boundary crossings and call-graph scope, yielding our current catalog of 54 distinct topological structures. While this set encompasses the core structural patterns dictated by conflict graph formation rules, we make no claim that the catalog is complete. The subsequent large-scale empirical analysis of our dataset serves to quantify the prevalence of these core structures and empirically validate their occurrence in real-world Java development.

●	Modification of the Left branch
●	Modification of the Right branch
●	Base code
□	File Boundary
→	Control/Data flow (method calls chains or field accesses)
---→	Interference

Fig. 3. Visual anatomy caption defining the structural components used in the topological structures.

In our visual anatomy of these structures (Figure 3), solid-border boxes represent distinct file boundaries. Circular nodes represent specific lines of code, color-coded to indicate their different branches of origin: green circles for modifications of the Left branch, magenta circles for modifications of the Right branch, and gray circles for the base code. Solid black arrows denote control or data flow across file boundaries (such

as method calls chains or field accesses), while dashed red arrows explicitly mark the interferences.

To formalize the creation and differentiation of the cataloged structures, we established a specific set of structural rules:

- 1) **Call abstraction:** Calls to the same file are not represented, and chained calls to different files are abstracted into a single call arrow between the initial and the final nodes. This ensures that only the modified lines and the final lines of call sequences appear in the structure.
- 2) **Branch symmetry:** Exchanging the roles of the Left and Right branch modifications (replacing green with magenta and vice versa) does not yield a distinct structure, as long as nodes originally belonging to the same branch continue sharing the same branch membership.
- 3) **Structure differentiation:** A structure A is considered distinct from a structure B if at least one of the following conditions is true:
 - (i) The number of files between A and B is different.
 - (ii) The number of call arrows between A and B is different.
 - (iii) The conflict endpoints are associated with different file partitions between A and B .

The structures are identified using an alphanumeric naming convention (e.g., A1, A2, B2, C3). In this notation, the numeric digit has a strict meaning: it indicates the exact number of distinct files involved in the conflict structure. Conversely, the letters (A, B, C, etc.) serve as arbitrary identifiers to distinguish the different structural permutations mapped and organized in our catalog.

Fundamentally, DF and OA structures exhibit a binary structural symmetry—differing primarily in the direction of their dependency flow. In DF, the flow propagates from a state element’s definition to its use; in OA, the relationship is inverted, originating from the overriding assignment towards the original definition. Conversely, CF structures present a wider topological variance. Because CF involves the convergence of multiple distinct flows into a single common point, they admit a wide range of configurations, reflecting the higher inherent complexity of confluence points.

In CF topologies, method calls originating from the base branch’s node (gray circles) are abstracted from the visual representation. Although a base line may invoke a method that computes or returns a value, creating an underlying flow and dependency across the code. This design choice reflects the base node’s strict purpose: to mark the point of confluence where independent contributions from the Left and Right branches intersect, rather than to depict the detailed call chains or execution context of the base code.

This analysis produced a catalog of topological structures, which we present as the first result in Section IV.

C. Dataset

To answer our research questions, we use the Ref-Dataset [6]³. This dataset comprises 907 merge scenarios from

295 open-source Java projects where two contributors modified the same method or constructor. We selected these units using the following criteria: GitHub repositories with at least 10 stars, using Maven or Gradle, and passing tests within 30 minutes on JDK 8, 11, or 17. An additional filter confirmed that the mutually modified target classes were included in the final JAR file.

For each scenario, we build the JAR file of the merge commit version without external dependencies. This reduces the amount of code analyzed, improving execution time, although it may decrease the accuracy of the static analysis tooling. We accept this trade-off because faster analysis is preferable for practical, everyday use of static analysis in detecting conflicts. Along with this, we collect the line numbers corresponding to the changes introduced by the Left and Right parent branches of the merged revision. By design, the chosen dataset restricts its scope to cases where two branches modified the same method, providing scenarios that are naturally more prone to behavioral conflicts. Although our analysis is interprocedural and traverses conflicts across multiple methods and files, this dataset restriction implies that the *origin* of every conflict is in changes to a single method. Consequently, our quantification cannot exercise topologies whose conflicting changes originate in distinct files, even though our catalog defines them.

D. Study Procedures and Instrumentation

We run the static analysis to detect DF, CF, and OA interferences for each experimental unit in the dataset. To capture interferences that span method boundaries, the analyses are executed interprocedurally using the Soot 4.5.0 program analysis framework [8] equipped with the SPARK pointer-analysis engine [9].

For each scenario, the analysis pipeline proceeds in four main steps. First, during bytecode loading, Soot loads the compiled JAR files for the merged versions. We execute our experiment with default settings for the Jimple Body Creation (jbc) phase of Soot, except for the `use-original-names` option, which preserves the original names of local state elements. We also configure Soot to keep line numbers from bytecode, which is essential to precisely map conflicts back to the source code.

Second, we leverage the SPARK framework to build a whole-program call graph using context-insensitive points-to analysis. To focus the analysis on developer-changed code, we configure Soot not to explore classes in standard Java libraries (e.g., `ArrayList` and `HashMap`), except for basic classes like `String`, `RuntimeException`, and wrapper classes for primitive types.

Third, during conflict detection, the analysis starts from each changed statement in Left or Right branches and traverses the call graph by recursively expanding callees, accumulating state writes and reads along the call chain. Recent work has investigated the impact of the analysis *depth limit* on semantic conflict detection, showing that while deeper limits (e.g., 10 levels) can capture additional conflicts, they also trigger timeout explosions and increase computational costs with

³<https://github.com/spgroup/ref-dataset>

diminishing returns [10]. Thus, following prior methodologies, we configure a depth limit of 5 levels from the analysis entry point to provide an optimal balance between execution time and detection accuracy.

Finally, in the topological classification step, the static analysis outputs a detailed report of the detected conflicts, which our procedure then uses to classify each conflict into a topological category. Our classification algorithm parses the stack traces provided in the report delivered by the static analysis, extracting only the origin and the outcome of the conflicting paths. Specifically, the algorithm crosses the reported execution stack with the modified lines, discarding the initial non-modified nodes so that the stack starts at the first modified line. Then, calls to the same file are not represented, and chained calls to different files are abstracted into a single call arrow between the initial and the final nodes, eliminating the redundancy of multiple consecutive calls. This abstraction ensures that only the modified lines and the last lines of the call sequences appear in the extracted topology. The exact size and shape of these two extracted file sequences determine the structural topology of the conflict, which is then matched against the reference topological structures defined in Section III-B.

To avoid variations in software configuration, we execute all experiments inside a single Docker container. The container runs on an Ubuntu machine equipped with an Intel Xeon Gold 6338 processor and 2 TB of RAM, ensuring consistent execution conditions.

We set the timeout to 300 seconds as the maximum time a user should reasonably wait for this type of analysis. If this time limit is exceeded, the analysis is interrupted.

IV. RESULTS

This section presents the results organized by our two research questions. We first characterize the topological structures for each conflict type (RQ1), then quantify the occurrence of each structure across the RefDataset (RQ2).

A. RQ1: Topological Structures of Semantic Conflicts

We systematically derived our catalog of topological structures by applying the structural differentiation rules defined in Section III-B to the formal conflict definitions from Section II. The derivation enumerates structurally distinct arrangements of the conflict entities across file boundaries and call-graph edges.

We organize the catalog along two primary dimensions of scope. The first is origin scope: whether the conflict *starts in the same file* (i.e., both the Left and Right modifications originate in a single class) or in different files. The second is convergence scope: whether the conflict *ends in the same file* (i.e., both the Left and Right flows converge into a single class) or whether it diverges to separate files. Combining these two binary properties yields four distinct, mutually exclusive topological groups: (i) same start file and same end file, (ii) same start file and different end file, (iii) different start file and same end file, and (iv) different start file and different end file.

Table I summarizes the resulting structure distributions across these four groups.

TABLE I
TOPOLOGICAL STRUCTURES BY ORIGIN AND CONVERGENCE SCOPE.

Conflict Type	Same Start File		Diff. Start File		Total
	Same End	Diff. End	Same End	Diff. End	
DF and OA	2	2	2	5	11
CF	4	7	10	11	32

The figures that follow use the visual notation defined in Figure 3. In the following subsections we discuss representative structures for each conflict type; the full catalog, containing detailed descriptions and code examples for all remaining structures, is available in our supplementary material [7].

1) *DF and OA Structures*: DF and OA conflicts share an identical catalog of 11 structures (Table I). Both analyses involve exactly two conflicting entities per branch; the only semantic difference is the direction of the dependency flow (definition-to-use for DF; write-to-write for OA). Because the topological classification depends solely on the number of files involved and the multi-file call-graph structure—not on the direction of dependency—the two catalogs are equivalent. Figure 4 presents all 11 structures organized by the number of distinct files involved (rows), with columns serving as arbitrary identifiers for the different structural permutations.

As shown in Table I, the 11 structures for DF and OA span all four topological groups. In the *Same Start File and Same End File* group, the intra-file structure **A1** serves as the baseline configuration where all modifications and conflicts occur within a single class, corresponding to the motivating examples in Listings 1 and 2. Divergent structures, such as **B2**, originate in a single file but cross file boundaries, separating the conflict points. Conversely, **C2** illustrates a convergent scenario where one modification propagates via a method call into the file where the other modification originates. Finally, fully scattered conflicts like **A3** involve distinct files where the conflict occurs independently across boundaries without any method calls between the modifications themselves. The full descriptions of the remaining seven structures (D2, C3, A2, E2, B3, D3, A4) are available in our online appendix [7].

2) *CF Structures*: CF conflicts involve three entities, the Left modification, the Right modification, and the base-version dependent expression, whose relative positions across file boundaries and call-graph edges determine the structure. This three-way dependency admits 32 distinct arrangements. The structural rules defined in Section III-B apply directly: structures G2 and G3 are distinct due to differentiation condition (i); C2 and G2 are distinct due to condition (ii); and G2 and I2 are distinct due to condition (iii). Figure 5 presents them organized by file count (rows), with columns as arbitrary identifiers. Among the 32 CF structures, four are intra-file (e.g., the baseline **A1**, corresponding to the motivating example in Listing 3), while the remaining 28 introduce varying degrees of scattering: **B2** converges in an external file; **A2**

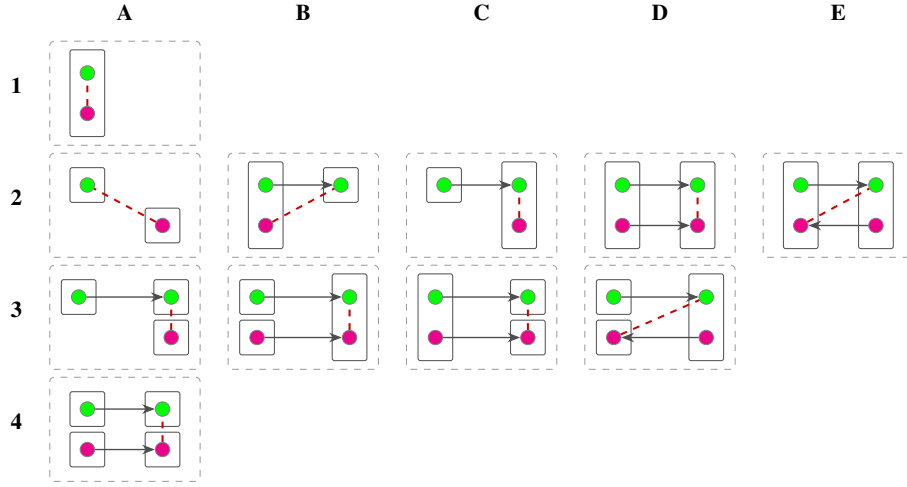


Fig. 4. Topological structures shared by DF and OA conflicts.

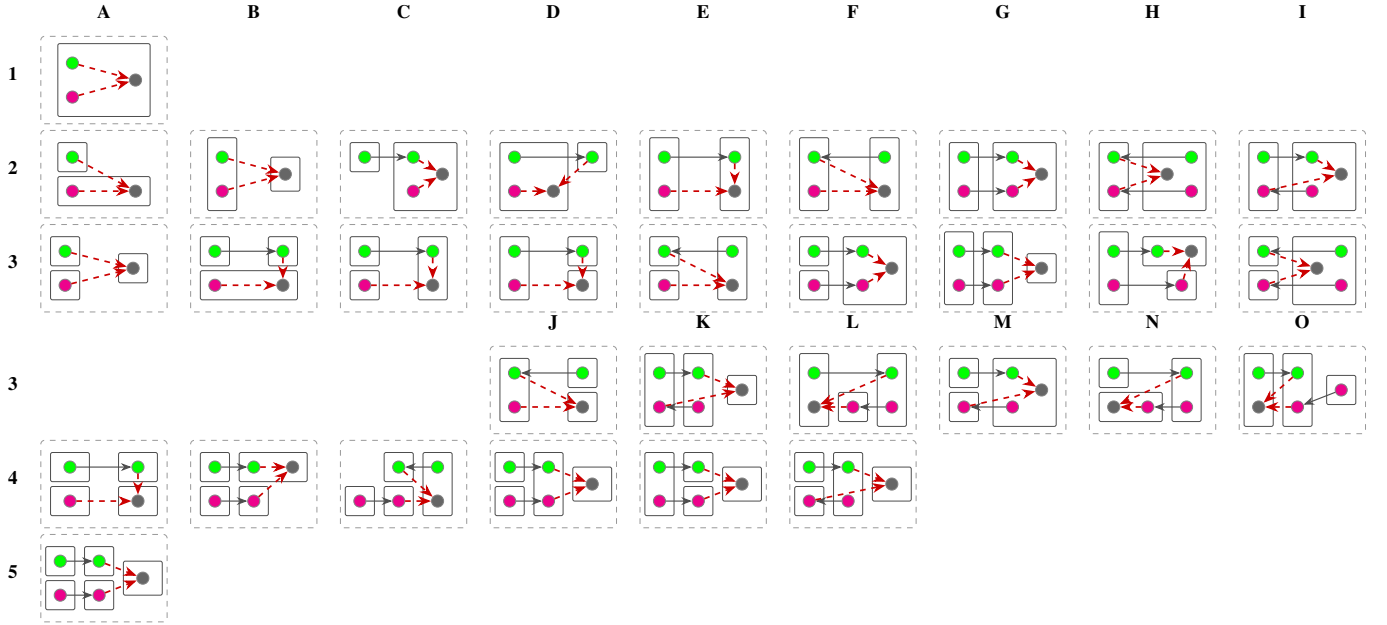


Fig. 5. Topological structures for CF conflicts.

originates across files but converges back into an origin file; and eleven are fully scattered, such as **A3**, whose modifications flow into a shared expression in a separate third class. The full descriptions are available in our online appendix [7].

RQ1

We derived a catalog of topological structures for semantic conflicts: 11 shared by DF and OA, and 32 for CF. Organized along origin and convergence scopes, it shows that although intra-file conflicts are structurally possible for all types, most structures exhibit some scattering, originating in different files, converging in different files, or both.

B. RQ2: Quantification of Conflict Occurrences

As stated in Section III-C, the observed distributions reflect only conflicts derived from changes to a single method. This is a restriction of the *dataset* (mutually modified methods), not of the interprocedural analysis itself, which spans multiple methods and files. Topologies whose conflicting changes originate in distinct files (e.g., A2, A3, A4) do not appear in our dataset.

Furthermore, our quantification excludes two specific scenarios that do not represent genuine semantic divergence: *unmodified stack traces* (where the stack trace leading to the conflict contains no modified methods, indicating the conflict does not propagate through modified code) and *symmetric modified lines* (where both branches independently apply the exact same modification to the exact same line, effectively resulting in no conflict).

We analyzed the occurrence of each topological structure to understand which patterns dominate in practice. Figures 6, 7, and 8 present the distributions for DF, OA, and CF conflicts based on our static analysis. For clarity, we display only the most frequent topologies.

For DF conflicts, excluding unmodified stack traces and symmetric modified lines, we successfully classified 31,747 instances. All 31,747 instances belong to structures where the modifications *start in the same file* (Groups 1 and 2), reflecting the constraint of our empirical dataset, which isolates conflicts originating from same-file modifications. Within this constraint, 60.3% of the occurrences belong to the *Same Start File and Different End File* group (Group 2), driven by divergent topologies like **B2** (10,947 occurrences, 34.5%) and **C3** (8,181 occurrences, 25.8%). The *Same Start File and Same End File* group (Group 1) accounts for the remaining 39.7%, distributed between the fully local **A1** structure (8,912 occurrences, 28.1%) and the multi-file **D2** structure (3,707 occurrences, 11.7%).

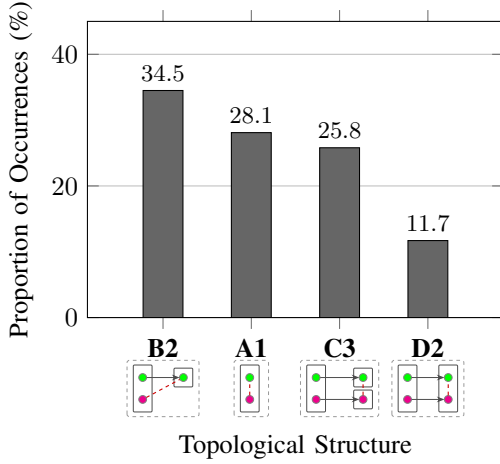


Fig. 6. Distribution of topological structures for DF conflicts.

For OA conflicts, excluding cases with symmetric modified lines and unmodified stack traces, we successfully classified 105 instances. Similar to DF, all instances originate from the same file due to dataset constraints. The *Same Start File and Same End File* group slightly edges out the divergent group, accounting for 53.3% of occurrences distributed between the fully local **A1** structure (32 occurrences, 30.5%) and the multi-file **D2** structure (24 occurrences, 22.9%). The *Same Start File and Different End File* group accounts for the remaining 46.7%, distributed among **C3** (31 occurrences, 29.5%) and **B2** (18 occurrences, 17.1%).

For CF conflicts, excluding unmodified stack traces and symmetric modified lines, we successfully classified 509 instances. Unlike DF and OA, CF conflicts are predominantly local: 90.4% belong to the *Same Start File and Same End File* group, dominated almost entirely by the fully local **A1** structure (458 occurrences, 90.0%), with a minor contribution from the multi-file **E2** structure (2 occurrences, 0.4%). The

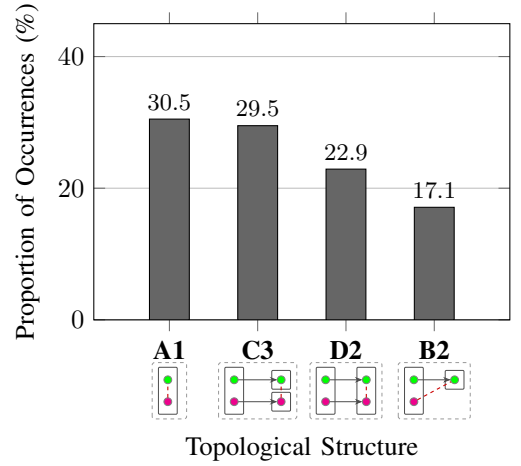


Fig. 7. Distribution of topological structures for OA conflicts.

Same Start File and Different End File group constitutes a minority (9.6%) of the conflicts, with the most frequent divergent topologies being **B2** (27 occurrences, 5.3%) and **G2** (19 occurrences, 3.7%). Two additional divergent structures appear at low rates: **D3** (2 occurrences, 0.4%) and **G3** (1 occurrence, 0.2%).

Structures D3 and E2 were not discussed as representatives in Section IV-A; we define them here for completeness. **D3** involves Left propagating from a first file to a second file; Right occurs directly in the first (origin) file; and the base-code dependent expression is located in a third distinct file. **E2** is a two-file structure where Left propagates via a method call from a first file to a second file, Right occurs directly in the first file, and the base-code dependent is on the second file.

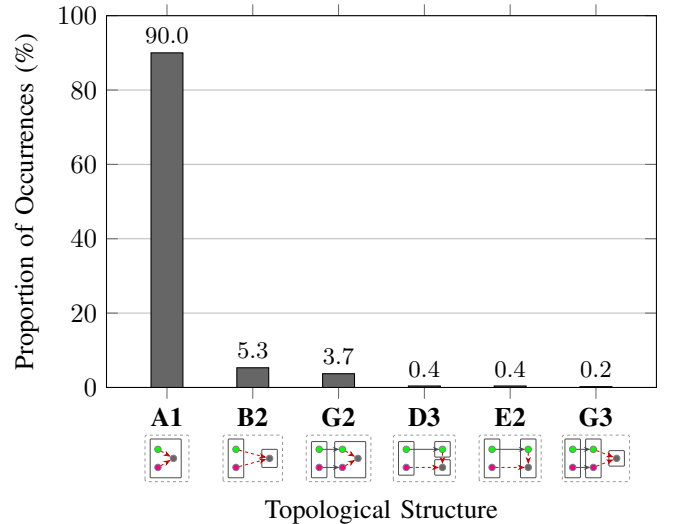


Fig. 8. Distribution of topological structures for CF conflicts.

This distribution highlights a fundamental dichotomy: while CF conflicts predominantly start and end within a single file

(Group 1), both DF and OA conflicts frequently diverge, starting in a single file but concluding their conflicts in separate classes (Group 2).

Since the entry point for the experimental units in our dataset is the method mutually modified by both branches, the origin of each interference is local by construction. Thus, the dichotomy reflects how far each conflict propagates from this shared origin, not where it begins. This propagation depends on the system’s organization; object-oriented principles, method decomposition, and architectural decisions can determine whether execution paths remain localized or cross class boundaries, shaping the topological distributions.

RQ2

The empirical distribution reveals a structural dichotomy: CF conflicts occur primarily as intra-file phenomena (Group 1), whereas DF and OA conflicts frequently diverge into different files (Group 2). Due to dataset limitations, all observed conflicts originated in the same file, but their convergence scopes vary by analysis.

V. DISCUSSION AND STUDY IMPLICATIONS

This section discusses the findings of our empirical characterization and outlines practical implications for software engineers, static analysis tool designers, and researchers based on the topological structures and their occurrences.

A. Implications for Practitioners and Tool Configuration

Our results provide concrete guidance for configuring static analysis tools in real-world integration pipelines. The structural dichotomy between CF conflicts and DF/OA conflicts suggests that practitioners should adopt tiered detection strategies. For CF conflicts, an intraprocedural analysis (or a shallow interprocedural analysis that does not cross file boundaries) is sufficient to detect nearly all instances, since 90.0% of them concentrate in the fully local **A1** structure. Conversely, for DF and OA conflicts, interprocedural analysis is essential—a purely intraprocedural check would miss not only the divergent *Same Start File and Different End File* group (60.3% of DF conflicts and 46.7% of OA conflicts) but also the multi-file instances from the *Same Start File and Same End File* group (such as **D2**, accounting for 11.7% of DF conflicts and 22.9% of OA conflicts). Practitioners who integrate semantic conflict detection into CI/CD pipelines can use this distinction to run fast, shallow CF checks on every pull request while reserving deeper, multi-file DF and OA analysis for nightly or pre-release builds where more computational resources are available.

B. Implications for Tool Designers

The concentration of CF conflicts in fully local topologies suggests that tool designers can optimize analysis performance by prioritizing intraprocedural paths before expanding

into multi-file call chains. A breadth-first traversal strategy—exhausting same-file callees before crossing file boundaries—would detect most CF conflicts at minimal computational cost. Furthermore, knowing that both DF and OA conflicts frequently exhibit divergent multi-file patterns like **B2**, **C3**, and **D2**, tools must specifically target these distributed call chains when tracking data flows and state element overrides, optimizing the search space for the most common distributed conflicts.

C. Implications for Researchers

The catalog of 54 topological structures (11 DF, 32 CF, 11 OA) presented in RQ1 provides a formal baseline for evaluating future semantic conflict detection techniques. Researchers can use these topological structures to synthesize test cases that ensure a new analysis tool can correctly identify the mapped structural variants, not just the fully local **A1** structure. The occurrence data from RQ2 highlights which structures are most critical for practical applicability, demonstrating that benchmarks must heavily emphasize divergent topologies for DF and OA.

Our study focuses exclusively on Java merge scenarios. The high locality of CF conflicts (**A1** dominance) and the distributed nature of DF and OA conflicts might generalize to other statically typed, object-oriented languages with similar scoping rules. However, languages with different dispatch mechanisms (for example, Python’s dynamic dispatch or JavaScript’s prototype chains) or different modularity paradigms (such as C’s header files) may exhibit entirely different topological distributions. Replicating this characterization study on multi-language benchmarks using our topological structure definitions constitutes a natural and necessary extension of our work.

VI. THREATS TO VALIDITY

A. Construct Validity

Construct validity concerns whether our measurement metrics reflect the intended concepts. Our topological classification relies on extracting stack traces from static analysis reports to determine file boundary crossings and structural relationships. While this approach accurately captures the structural properties of conflicts detected by the analysis, it may miss semantically equivalent conflicts that operate through complex cross-method aliasing patterns not fully resolved by SPARK’s context-insensitive points-to analysis. Furthermore, SPARK builds a whole-program call graph, pruning unreachable edges; in projects that rely heavily on runtime class loading, this excludes call paths that are in fact reachable at run time. Additionally, Soot and SPARK possess known limitations in resolving Java reflection, dynamic proxies, and lambda expressions, which may result in missed conflicts inside code paths that use these features. Finally, our catalog of topological structures was derived empirically; although it covers the core patterns implied by our structural rules, we have no proof that it enumerates every possible configuration.

B. Internal Validity

Internal validity considers confounding factors that might influence our results. A primary internal threat relates to the analysis configuration parameters, specifically the 5-level call-graph depth limit and the 300 seconds per-scenario timeout boundary imposed during execution. While the depth limit provides a necessary balance between execution time and detection accuracy, it inherently prevents the discovery of semantic conflicts that manifest across extremely deep call chains. Furthermore, complex, pathological call graphs risk triggering timeouts before the analysis completes its traversal. While these boundaries may truncate some exceptionally distributed topologies (such as D3 or G3), prior research [10] demonstrates that a 5-level limit effectively captures the most significant semantic interference patterns while avoiding diminishing returns. Consequently, our reported distributions provide evidence of structural patterns that occur in the analyzed dataset and may be representative of patterns encountered in software development practice.

C. External Validity

External validity addresses the generalizability of our findings. We sourced our merge scenarios exclusively from popular open-source Java projects hosted on GitHub, filtered by build success and the presence of textual conflicts. Consequently, the observed topologies may reflect the specific modularity and encapsulation practices common in open-source Java development, rather than a universal law of software engineering. Crucially, as highlighted in Section IV-B, our dataset strictly comprises scenarios where both branches modified the exact same method. This design choice inherently constrains the origin of semantic changes to a single file, effectively preventing the manifestation of topologies where conflicting changes originate in distinct files (such as A2, A3, or A4). While this limitation represents a threat to external validity, it stems from the available dataset and tooling rather than from the interprocedural analyses, which itself spans multiple methods and files. Finally, our findings may not transfer directly to closed-source enterprise projects with different architectural conventions, unmaintained codebases with irregular merge histories, or systems written in languages with different dispatch and scoping mechanisms (e.g., Python or C++).

VII. RELATED WORK

This section positions our quantitative characterization study within the broader context of collaborative development research, highlighting approaches that identify, classify, and mitigate code integration conflicts. We organize the discussion into four categories and explicitly contrast each line of work with our contributions.

A. Empirical Studies on Merge Conflicts

Researchers extensively investigate the challenges of collaborative software development and code integration. Ghiotto et al. [11] investigate the nature of textual merge conflicts

across 2,731 open-source Java projects, categorizing the typical syntax clashes developers face daily and quantifying their frequency and resolution complexity. Accioly et al. [12] explore the characteristics of semi-structured merge conflicts, analyzing code composition patterns and structural properties of conflicting regions. Elias et al. [13] assess merge tools and strategies, comparing resolution recommendations that assist developers in handling textual integration failures. Cavalcanti et al. [2] evaluate and improve semistructured merge techniques, demonstrating that structure-aware merging reduces the incidence of false conflicts.

The closest empirical study to our work is by Da Silva et al. [5], who investigate the frequency, causes, and resolution patterns of build conflicts in open-source Java projects. While they provide a catalogue of build conflict causes, highlighting that missing declarations cause 65% of build conflicts (for example, *Unavailable Symbol*), their focus is strictly on conflicts that break the build process. Similarly, researchers investigate build conflict frequency and early detection. For example, Kasi and Sarma [14] and Brun et al. [3], [15] report on the frequency of build conflicts to motivate proactive conflict minimization and speculative analysis. In an industrial context, Sung et al. [16] explore build conflicts that arise when integrating changes across forks, proposing strategies for understanding and fixing them.

Unlike these studies, which focus on textual, syntactic, or build conflicts that produce visible merge warnings or compilation errors, our work targets dynamic semantic conflicts, behavioral conflicts that occur silently even when textual merges and builds succeed without any warnings. We complement the textual and build conflict literature by providing the first catalog of topological structures for DF, CF, and OA conflicts and quantifying their occurrences.

B. Semantic Conflict Detection

Since dynamic semantic conflicts arise from changes in program behavior, researchers traditionally rely on dynamic execution and test generation to detect them. Da Silva et al. [17] propose an automated behavior change detection technique that uses unit test generation to identify semantic conflicts arising from code integration. Brun et al. [3], [15] introduce speculative analysis for proactive conflict detection, executing tentative merges in the background and running project test suites to identify integration failures before they reach the main branch. However, the quality and coverage of the underlying test suite limit test-based methods, which also incur substantial computational costs when run repeatedly. Our study provides the topological structure data that test-based approaches could use to prioritize which structural patterns warrant the most test generation effort.

To overcome the limitations of dynamic execution, other researchers address semantic conflicts through formal methods and static analysis. Horwitz et al. [18] introduce Program Dependency Graphs (PDGs) and System Dependency Graphs (SDGs) to integrate non-interfering program versions, providing semantic guarantees about merge correctness. Sousa et

al. [19] propose SafeMerge, a formal verification tool that uses program verification techniques to prove the absence of unwanted merge behaviors. Although theoretically sound, these techniques face scalability challenges on large codebases. Our study operates at a different abstraction level: rather than proposing a new analysis algorithm, we provide a structured catalog of the topological structures that these tools aim to detect.

The closest works regarding static semantic conflict detection are by Santos de Jesus et al. [4] and Barbosa et al. [20]. Santos de Jesus et al. compare static analyses for improved semantic conflict detection, evaluating DF, CF, and OA analyses across multiple pointer-analysis configurations, while Barbosa et al. investigate the effect of different call-graph construction algorithms on the precision of OA analysis. While they focus on comparing detection accuracy and pointer-analysis sensitivity, our study directly complements their research by systematically formalizing the structures of these conflicts and quantifying how often each specific topology occurs in practice.

C. AI and Machine Learning for Conflict Resolution

Researchers apply Artificial Intelligence (AI) and Machine Learning (ML) to detect and resolve merge conflicts. They leverage deep learning models to resolve textual conflicts. For instance, Svyatkovskiy et al. [21] formulate program merge conflict resolution via neural transformers, treating the problem as a classification task over three-way diffs. Similarly, Dinella et al. [22] propose DeepMerge, a pointer network-based architecture that learns to merge programs by rearranging existing code. Dong et al. [23] further explore this by proposing generative models that can synthesize new tokens to resolve conflicts, rather than merely classifying existing ones. Other studies combine heuristics with Large Language Models (LLMs); Shen et al. [24] use lightweight classifiers for simple strategies and rely on ChatGPT for complex resolution scenarios, while Zhang et al. [25] demonstrate that pre-trained language models can effectively resolve real-world textual and static semantic conflicts. Furthermore, researchers investigate the use of LLMs for detecting semantic conflicts [26] and evaluating their robustness in code generation tasks [27].

While these AI-based techniques show promising results in automating the resolution of textual and build conflicts, our study diverges by focusing on the static detection and characterization of dynamic semantic conflicts. Generative AI approaches still face challenges related to context window limits, hallucinations, and a reliance on compilation errors to locate issues. By contrast, our work contributes a foundational characterization of the structural topologies (DF, CF, and OA) that lead to silent behavioral conflicts, which can ultimately inform and improve the prompts, context selection, and training data for future AI-driven resolution tools.

VIII. CONCLUSION

This paper presented a systematic topological characterization of semantic conflicts in open-source Java projects. We

defined a catalog of 54 structures that capture how DF (11), CF (32), and OA (11) conflicts manifest across file boundaries and call graphs. We then quantified the occurrence of each structure by executing an interprocedural static analysis on 907 real-world merge scenarios from the RefDataset, detecting 32,361 conflict instances. Since the dataset restricts to merge scenarios where the merged branches modify the same method, the conflicting changes necessarily originate in the same file, while the catalog itself remains general.

Our study revealed a structural dichotomy in semantic conflicts: CF conflicts concentrate in intra-file topologies (with 90.0% of CF instances belonging to the single-file A1 structure), while both DF and OA conflicts exhibit highly divergent or inter-file behavior, with topologies that start in the same file but end in different files accounting for 60.3% and 46.7% of their respective instances. This structural dichotomy demonstrates that different analyses require fundamentally different detection strategies: an intraprocedural analysis is sufficient to detect almost all CF conflicts, whereas detecting DF and OA conflicts necessitates interprocedural capabilities across different files.

These findings offer actionable guidance for practitioners configuring static analysis tools in daily software integration pipelines, suggesting tiered detection strategies that reserve deeper, multi-file traversal specifically for DF and OA patterns.

ARTIFACT AVAILABILITY

Our dataset, analysis scripts, topological structure definitions, and raw conflict detection outputs are available in our online appendix to support reproducibility and replication [7].

ACKNOWLEDGEMENTS

For partially supporting this work, we would like to thank INES.IA (National Institute of Science and Technology for Software Engineering Based on and for Artificial Intelligence) www.ines.org.br, and CNPq grants 408817/2024-0 and 401032/2025-6. We also thank the members of the Software Productivity Group for their valuable feedback, insightful suggestions, and overall support throughout this research.

REFERENCES

- [1] S. Khanna, K. Kunal, and B. C. Pierce, "A formal investigation of diff3," in *Foundations of Software Technology and Theoretical Computer Science*, 2007.
- [2] G. Cavalcanti, P. Borba, and P. Accioly, "Evaluating and improving semistructured merge," *Proceedings of the ACM on Programming Languages*, 2017.
- [3] Y. Brun, R. Holmes, M. D. Ernst, and D. Notkin, "Early detection of collaboration conflicts and risks," *IEEE Transactions on Software Engineering*, 2013.
- [4] G. Santos de Jesus, P. Borba, R. Bonifácio, and M. Barbosa, "Comparing static analyses for improved semantic conflict detection," *Automated Software Engineering*, 2025.
- [5] L. Da Silva, P. Borba, and A. Pires, "Build conflicts in the wild," *Journal of Software: Evolution and Process*, 2022.
- [6] V. Lira, P. Borba, R. Bonifácio, G. Santos de Jesus, and M. Barbosa, "Reffilter: improving semantic conflict detection via refactoring-aware static analysis," 2025.
- [7] "Online appendix," <https://spgroup.github.io/papers/SCAM2026/index.html>, 2026.

- [8] R. Vallée-Rai, P. Co, E. Gagnon, L. Hendren, P. Lam, and V. Sundaresan, "Soot: a java bytecode optimization framework," in *Conference of the Centre for Advanced Studies on Collaborative Research*, 2010.
- [9] O. Lhoták and L. Hendren, "Scaling java points-to analysis using SPARK," in *International Conference on Compiler Construction*, 2003.
- [10] P. Patriota, M. Barbosa, and P. Borba, "The impact of static analysis depth limit on semantic conflict detection," in *Brazilian Symposium on Software Engineering*, 2026.
- [11] G. Ghiotto, L. Murta, M. d. O. Barros, and A. van der Hoek, "On the nature of merge conflicts: a study of 2,731 open source java projects hosted by github," *IEEE Transactions on Software Engineering*, 2020.
- [12] P. Accioly, P. Borba, and G. Cavalcanti, "Understanding semi-structured merge conflict characteristics in open-source java projects," *Empirical Software Engineering*, 2018.
- [13] P. Elias, H. d. S. Campos Junior, E. Ogasawara, and L. G. P. Murta, "Towards accurate recommendations of merge conflicts resolution strategies," *Information and Software Technology*, 2023.
- [14] B. K. Kasi and A. Sarma, "Cassandra: proactive conflict minimization through optimized task scheduling," in *International Conference on Software Engineering*, 2013.
- [15] Y. Brun, R. Holmes, M. D. Ernst, and D. Notkin, "Proactive detection of collaboration conflicts," in *Foundations of Software Engineering*, 2011.
- [16] C. Sung, S. K. Lahiri, M. Kaufman, P. Choudhury, and C. Wang, "Towards understanding and fixing upstream merge induced conflicts in divergent forks: an industrial case study," in *International Conference on Software Engineering*, 2020.
- [17] L. Da Silva, P. Borba, W. Mahmood, T. Berger, and J. a. Moisakis, "Detecting semantic conflicts via automated behavior change detection," in *International Conference on Software Maintenance and Evolution*, 2020.
- [18] S. Horwitz, J. Prins, and T. Reps, "Integrating noninterfering versions of programs," *ACM Transactions on Programming Languages and Systems*, 1989.
- [19] M. Sousa, I. Dillig, and S. K. Lahiri, "Verified three-way program merge," in *Proceedings of the ACM on Programming Languages*, 2018.
- [20] M. Barbosa, P. Borba, R. Bonifácio, V. Lira, and G. Santos de Jesus, "The effect of pointer analysis on semantic conflict detection," 2025.
- [21] A. Svyatkovskiy, S. Fakhoury, N. Ghorbani, T. Mytkowicz, E. Dinella, C. Bird, J. Jang, N. Sundaresan, and S. K. Lahiri, "Program merge conflict resolution via neural transformers," in *Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022.
- [22] E. Dinella, T. Mytkowicz, A. Svyatkovskiy, C. Bird, M. Naik, and S. K. Lahiri, "DeepMerge: learning to merge programs," *IEEE Transactions on Software Engineering*, 2023.
- [23] J. Dong, Q. Zhu, Z. Sun, Y. Lou, and D. Hao, "Merge conflict resolution: classification or generation?" in *International Conference on Automated Software Engineering*, 2024.
- [24] C. Shen, W. Yang, M. Pan, and Y. Zhou, "Git merge conflict resolution leveraging strategy classification and LLM," in *International Conference on Software Quality, Reliability, and Security*, 2023.
- [25] J. Zhang, T. Mytkowicz, M. Kaufman, R. Piskac, and S. K. Lahiri, "Using pre-trained language models to resolve textual and semantic merge conflicts (experience paper)," in *International Symposium on Software Testing and Analysis*, 2022.
- [26] L. Sun, Y. Lu, X. Mao, T. Zhang, Z. Zhang, and H. Zhou, "ConflictLens: an LLM-based method for detecting semantic merge conflicts," in *International Conference on Software Engineering and Knowledge Engineering*, 2025.
- [27] J. Liu, S. Xu, Y. Yao, Y. Shen, Y. Zhao, X. Zhu, P. Yu, F. Xu, and X. Ma, "Robustness evaluation and enhancement of LLMs in code generation: an empirical study," *Empirical Software Engineering*, 2026.